

The Self-Hosted Runner

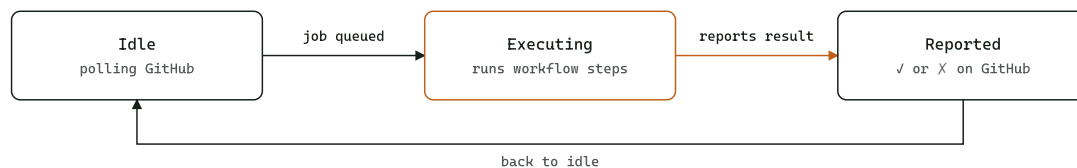
A small agent process that executes workflow steps. "Self-hosted" means it runs on your infrastructure instead of GitHub's — here, on clubuntu itself.

concept doc — no execution in this step 2026-08-16

01 It polls; it isn't pushed to

The single most important thing about a runner: GitHub never dials into it. The runner dials *out* to GitHub — a long-lived outbound connection asking "anything queued for me?" — and GitHub responds with a job when one exists. That direction is what makes running this on clubuntu safe: nothing new has to be opened or exposed for GitHub to reach it.

02 Lifecycle



Between jobs, the runner is just a quiet background process — no listening port, no inbound connection to defend.

03 The privilege it needs

To run `docker compose build/push/pull/up`, the runner's user needs membership in the `docker` group on clubuntu — which is effectively root-equivalent on that host, the same caveat already documented for `docker-mcp`'s bind-mounted socket. This isn't a flaw specific to the runner; it's inherent to anything that drives the Docker daemon directly.

Run it as a dedicated, non-login user added only to the `docker` group — not your own account, not root directly. Scopes the blast radius to "can control Docker," not "can do anything you can."

04 Registering it

- ▶ GitHub repo → Settings → Actions → Runners → New self-hosted runner
- ▶ Copy the one-time registration token it generates
- ▶ On clubuntu: download the runner, run its `config.sh` with that token
- ▶ Install it as a systemd service (`svc.sh install && svc.sh start`) so it survives reboots

This step happens on GitHub's UI and on clubuntu directly — outside what this session can do; exact commands to paste once the repo exists.